

Satellite TV System Design for MITRE eCTF 2025

JumByte from Tufts University

1 Overview

This design document describes Tufts University's implementation of a Satellite TV system, specifically focusing on the encoder and decoder of the system. Our design employs a multi-layer security approach combining hardware-optimized cryptography via WolfSSL and robust key management. Our implemented security principles include:

Encryption We utilized two layers of AES symmetric encryption to secure our satellite.

Authentication Our authentication process ensures that tampered messages are rejected by the decoder, utilizing SHA-256 for hashing and HMAC SHA-256 to ensure message integrity.

Forward Secrecy Ephemeral session keys for each channel are utilized to ensure forward secrecy.

Replay Prevention Monotonically increasing timestamps prevent unauthorized viewers from replaying frames to which they should not have access.

Secure Authorization and Access Control Subscription-based access control prevents unauthorized users from viewing channels they are not subscribed to.

Memory Safety and Buffer Overflow We enforce memory safety through the use of MPU (Memory Protection Unit) guard regions to prevent buffer overflows and unauthorized memory access. Additionally, all buffer operations include explicit bound checking to mitigate memory corruption vulnerabilities. UART input is validated to prevent buffer overflows, and our system follows secure memory allocation practices to prevent stack and heap overflows.

2 Team Overview

We are a team of 19 undergraduate and graduate students participating in the 2025 MITRE eCTF challenge at Tufts University in the Spring 2025 semester. The team is advised by professors Ming Chow and Steven Bell.

3 System Architecture

The satellite TV system consists of the following main components:

- **Uplink:** Initiates TV frame transmission
- **Encoder:** Secures TV frame using encryption and hashing
- **Satellite:** Broadcasts encoded frames to receivers
- **Decoder (MAX7800FTHR):** Decrypts and validates received frames
- **Host Computer & TV:** Displays decoded content

The data flows from the Uplink to the Encoder, where it is secured. The secured frames are then transmitted by the satellite to the Decoder. The Decoder authenticates and decrypts the frames before sending them to the Host Computer & TV for display.

4 Cryptographic Primitives

This section outlines the various cryptographic techniques implemented throughout this design process to ensure that the frames and other data that are transmitted via the satellite TV system are secure, confidential, and authenticated. All implementations and their dependencies were open-source and proven to be resistant to commonly known vulnerabilities.

4.1 Encryption Primitives

4.1.1 Cryptography Open Source Project

Our implementation leverages the open-source Cryptography library within Python to handle encryption, authentication, and key management in the encoder. This library provides high-level APIs for secure cryptographic operations and low-level interfaces for fine-grained control over primitives such as AES (GCM and CBC), HMAC-SHA256, and HKDF. Specifically, we used the Cryptography package to:

- Perform AES-GCM encryption for frame payloads, ensuring both confidentiality and integrity.
- Perform AES-CBC encryption for system-wide protection, preventing meta-data leakage.
- Generate HMAC-SHA256 authentication codes to authenticate frame integrity and prevent tampering.
- Use HKDF key derivation to securely derive per-channel encryption keys.

For more details, visit the [Cryptography Project](#)

4.1.2 WolfSSL

While we utilized the Cryptography library to handle encryption, the decryption was handled by WolfSSL, which provided a faster and more lightweight framework for our embedded system. Specifically, we used WolfSSL on the decryption side to:

- Utilize HMAC-SHA256 to verify the message's integrity before initializing decryption.
- Utilize AES-GCM decryption to decrypt the inner payload.
- Utilize AES-CBC decryption to decrypt the outer encryption layer.
- Generate cryptographically secure IVs and nonces.
- Ensure safe memory allocation and handling for key storage and other cryptographic operations.

For more information, visit wolfssl.com

4.1.3 AES-GCM

Advanced-Encryption-Standard - Galois/Counter Mode is a widely used cryptographic algorithm that utilizes symmetric encryption. The Galois/Counter mode adds authentication to ensure integrity.

Main Components

- **Key:** The AES-GCM encryption method utilizes a symmetric key, either 128, 192, or 256 bits in size.
- **Initialization Vector (IV):** A unique, random value ensures that each iteration of plaintext has a different ciphertext (even if the plaintext is the same). The IV must be unique for every encryption under the same key. If an IV is ever reused with the same key, an attacker could potentially deduce relationships between plaintexts. This could ultimately lead to a loss of confidentiality. To ensure security, it is essential that a secure method, such as using a cryptographically secure random number generator, is used to generate these IVs.
- **Authentication Tag:** A fix-sized tag for authentication that ensures limited tampering with the data.

Encryption Process: First, a symmetric key and unique IV are generated. The AES-GCM encryption method uses counter-mode (CTR) where it encrypts a counter-value to produce a keystream. This keystream is then XORed with the plaintext to produce the ciphertext. GHASH, a polynomial hashing function in $GF(2^{128})$, is then implemented to compute the authentication tag, ensuring ciphertext integrity. GHASH internally performs Galois field multiplication, treating ciphertext blocks as coefficients in a polynomial.

Key Expansion and Counter Mode

1. A counter block (denoted as X_i) is generated for each block of plaintext.
2. The counter block is **encrypted using AES** with the key K :

$$E_K(X_i) = \text{AES-Encrypt}(K, X_i)$$

3. The ciphertext block is produced by XORing the plaintext block with the AES-encrypted counter:

$$C_i = P_i \oplus E_K(X_i)$$

Galois Field Authentication

1. A hash subkey H is derived by encrypting an all-zero block:

$$H = E_K(0^{128})$$

2. The authentication tag is computed using GHASH, which is a polynomial evaluation in $GF(2^{128})$:

$$\text{GHASH}(H, A, C) = \left(\sum_{i=1}^m C_i \cdot H^{m-i+1} \right) \oplus \left(\sum_{j=1}^n A_j \cdot H^{n-j+1} \right)$$

where:

- C_i are ciphertext blocks
- A_j are associated data blocks (AAD)
- H is the hash key derived from AES encryption of zero input

Final Authentication Tag

$$T = E_K(J_0) \oplus \text{GHASH}(H, A, C)$$

where J_0 is the pre-counter block derived from the IV.

Decryption Process: To decrypt, the recipient checks the authentication tag to verify that the encrypted plaintext has not been tampered with. CTR is then used to decrypt the ciphertext, and the plaintext output is recovered.

1. The ciphertext is decrypted using AES-CTR mode:

$$P_i = C_i \oplus E_K(X_i)$$

2. The authentication tag is recomputed using GHASH and verified against the received tag T . If the values do not match, decryption fails.

4.1.4 AES-CBC

Advanced Encryption Standard - Cipher Block Chaining is a widely used encryption method for block ciphers. Unlike AES-GCM, CBC does not include authentication (unless it is paired with HMAC).

Main Components

- **Key:** AES-CBC utilizes a symmetric key, either 128, 192, or 256 bits long.
- **Initialization Vector (IV):** A unique, random value ensures that each iteration of plaintext has a different ciphertext (even if the plaintext is the same).
- **Padding:** Since AES has size requirements, if the last block of plaintext is not 16 bytes, it must be padded. A common padding protocol is PKCS#7, which adds bytes equal to the number of missing bytes.

Encryption Process: First, a secret key is generated along with an initialization vector. AES-CBC chains blocks together by XORing each plaintext block with the previous ciphertext block before encryption. Each block therefore depends on the previous one, making it secure but potentially slower, since it cannot be parallelized.

1. The first plaintext block is XORed with an initialization vector (IV) before being encrypted:

$$C_1 = E_K(P_1 \oplus IV)$$

2. Each subsequent plaintext block is XORed with the previous ciphertext block before AES encryption:

$$C_i = E_K(P_i \oplus C_{i-1}) \quad \text{for } i \geq 2$$

where:

- E_K is the AES encryption function with key K .
- P_i represents the plaintext blocks.
- C_i represents the ciphertext blocks.
- IV is the initialization vector, a random value required to ensure uniqueness in encryption.

Decryption Process: Unlike the AES-GCM encryption algorithm, authentication is not involved (unless it is paired with HMAC), so the decryption process merely involves using AES to decrypt each block and then XORing it with the previous ciphertext. Once padding is removed, the original plaintext can be recovered.

1. The first plaintext block is recovered by decrypting the first ciphertext block and XORing it with the IV:

$$P_1 = D_K(C_1) \oplus IV$$

2. Each subsequent plaintext block is recovered by decrypting the ciphertext and XORing it with the previous ciphertext block:

$$P_i = D_K(C_i) \oplus C_{i-1} \quad \text{for } i \geq 2$$

where:

- D_K is the AES decryption function with key K .
- C_i represents the ciphertext blocks.
- P_i represents the plaintext blocks.
- IV is the same initialization vector used during encryption.

After, decryption padding is removed to recover the original message.

4.1.5 HMAC-SHA256

Hashed-based Message Authentication Code using SHA256 ensures the integrity of the data being passed.

Main Components

- **Key:** A secret key
- **Cryptography Hash Function:** SHA256 is used in this particular instance.

The HMAC Function The HMAC function is defined as:

$$\text{HMAC}(K, M) = \text{SHA-256}((K_{\text{opad}} \oplus \text{SHA-256}(K_{\text{ipad}} \oplus M)))$$

where:

- K is the secret key.
- M is the message to authenticate.
- K_{ipad} is the inner padded key.
- K_{opad} is the outer padded key.
- $\oplus$ represents bitwise XOR.

Key Padding and Block Size SHA-256 operates on 512-bit (64-byte) blocks, so the key K must be padded or hashed to match this length.

1. If K is longer than 64 bytes, it is hashed first:

$$K = \text{SHA-256}(K)$$

2. If K is shorter than 64 bytes, it is padded with zeroes:

$$K = K || 0x00 \text{ (repeated until 64 bytes)}$$

Inner Hash Computation The inner hash is computed by XORing the padded key with the inner padding constant (ipad = $0x36$) and hashing the result:

$$K_{\text{ipad}} = K \oplus (0x36 \times 64)$$

$$\text{Inner Hash} = \text{SHA-256}(K_{\text{ipad}} || M)$$

where:

- K_{ipad} is 64 bytes long.
- M is the **message being authenticated**.

Outer Hash Computation The outer hash is computed by XORing the padded key with the outer padding constant (opad = $0x5C$) and hashing the inner hash:

$$K_{\text{opad}} = K \oplus (0x5C \times 64)$$

$$\text{HMAC-SHA256} = \text{SHA-256}(K_{\text{opad}} || \text{Inner Hash})$$

Final Computation The final output is a 256-bit (32-byte) authentication tag, which can be used to validate message integrity:

$$TAG = \text{HMAC-SHA256}(K, M)$$

Process: The fixed 32-byte authentication tag is appended to encrypted data like a fingerprint. When the data is received, a decoder extracts the authentication tag and recomputes the HMAC-SHA256 using the IV and ciphertext. It then compares the computed to the extracted to determine authenticity.

4.2 Key Derivation and Randomness Primitives

4.2.1 HKDF

HMAC-Based Key Derivation Function is a secure method used to derive unique encryption keys from a master key. One of the key elements of HKDF in preventing attacks is enhancing key separation. Because HKDF derives unique, independent keys from one master key, it ensures that compromising one key does not initiate the downfall of the remaining keys and the attacker will only be able to decrypt limited portions of data.

HKDS consists of two main phases:

- **Extract Phase:** Converts potentially weak input key material into a stronger, fixed-length pseudorandom key.
- **Expand Phase:** Uses this pseudorandom key to generate multiple cryptographic keys with desired properties.

Extract Phase (PRK Calculation) The extract phase takes in:

- IKM = Input Keying Material (the shared secret or seed)
- $salt$ = Optional salt value (randomly generated or provided)

It then computes a pseudorandom key (PRK) using HMAC-SHA256:

$$PRK = \text{HMAC-SHA256}(salt, IKM)$$

Note:

- If salt is not provided, HKDF uses a default all-zero string.
- The resulting PRK is always 256 bits (32 bytes) when using SHA-256.

Expand Phase The expand phase generates multiple cryptographic keys from the PRK. This is done using the following iterative process:

$$T_1 = \text{HMAC-SHA256}(PRK, info || 0x01)$$

$$T_2 = \text{HMAC-SHA256}(PRK, T_1 || info || 0x02)$$

$$T_n = \text{HMAC-SHA256}(PRK, T_{n-1} || info || n)$$

where:

- PRK is the pseudorandom key from the extract phase.
- info is optional application-specific context (e.g., protocol identifier).
- T_n are intermediate values used to generate output keys.
- The final keying material is computed as:

$$OKM = T_1 || T_2 || \dots || T_L$$

where L is the desired output key length.

5 Software-Side Encryption & Decryption Security

5.1 Considered Threats and Mitigation Summary

Prior to initializing our design process, we evaluated potential threats our system might face from attackers and identified mitigation protocols to safeguard against intrusions.

Security Threat	Mitigation	Implemented Protocol
Expired Subscription	Check current timestamp and compare it with the start and end timestamp of subscription	Range check
Replay Attacks	Check if incoming timestamp is increasing by comparing with the last timestamp	Range check
Pirated Subscription	Verify if decoder ID and signature match	Value check and HMAC-SHA256
No subscription (Man in the middle attack)	Encryption of channel number and timestamp can help to prevent metadata leakage even when data is intercepted.	AES-CBC
Tampering of Frames	Authentication tag and HMAC-SHA256 ensures data integrity by detecting modifications.	AES-GCM and HMAC-SHA256
Key Collision Attack	Reuse of keys could lead to attackers uncovering key values and decrypting sensitive data. Various protocols have been implemented to prevent this.	Random Nonce and IV, HKDF, HMAC
Brute Force Attacks	Use of strong cryptographic algorithms and protocols to resist brute force attacks.	AES Encryption and HMAC-SHA256

Table 1: Security Threats and Mitigations

5.2 Building Our Security Strategy

Once we identified possible threats, we were able to construct a more robust strategy to design a secure system that could withstand a wide variety of attacks.

5.2.1 Two-Layer Strategy For Unauthorized Decoding Prevention

To prevent unauthorized decoding of TV frames, our implementation employs a robust two-layer encryption approach using the Python cryptography package and decryption using WolfSSL. The system encrypts each frame twice: first in an AES-GCM channel-specific encryption layer and then a second time in an AES-CBC system-wide encryption layer that employs a master key. This two-layer approach ensures that even if an attacker decrypts the outer layer, they will still need the channel-specific key to decrypt the payload.

- **Layer 1: The Inner Layer** The inner encryption layer provides protection for the frame payload using AES-GCM. Each channel has a unique encryption key, which is derived from the master key using HKDF.

Key Security Considerations For Encrypting The Frame Payload

- **Data Integrity and Protection:** The encryption method used in the inner layer, AES-GCM provides both encryption and authentication.
- **Using Randomness To Enhance Security:** Each frame is uniquely encrypted using a randomly generated nonce. This means that even if two frames contained identical data, they will still have their own unique encryption. This is critical for preventing replay attacks.

- **Using Channel Isolation To Enhance Security:** To add an additional layer of security, each channel is uniquely encrypted with its own key, which is derived from the master key using HKDF. This means that if attackers were to break the key for one channel, they would still be unable to break it for the remaining channels.

Inner Layer Encryption Process: The inner layer encryption process is implemented primarily in encryption.py where a per-channel encryption key is derived from the master AES-256 key using HKDF in gen_secrets.py. When the frame is encrypted, a random 12-byte nonce is generated using os.urandom(12), which ensures each encryption is unique even if the data is identical. The AES-GCM encryption process combines this nonce, the per-channel key and the frame payload to produce a ciphertext and a 16-byte authentication tag, which protects the payload from tampering. The final output is then passed to the outer encryption layer for further protection.

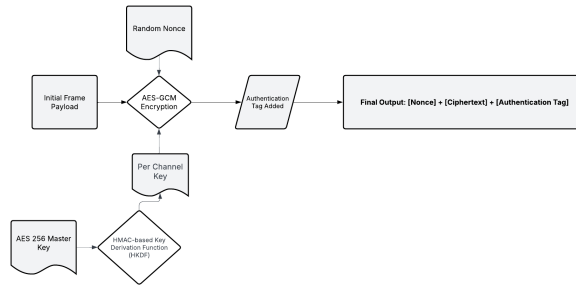


Figure 1: Inner-Layer Encryption Strategy

- **Layer 2: The Outer Layer** The outer encryption layer secures the entire frame structure, which includes
 - **Channel Number:** Used to identify the requested channel
 - **Timestamp:** Used to verify if a channel is active and prevent replay attacks
 - **AES-GCM Encrypted Payload:** The inner layer encrypted earlier

Unlike our inner layer, our outer layer is encrypted using AES-CBC with a system-wide master key.

Key Security Considerations For Encrypting Our Outer Layer:

- **Metadata Protection:** While it's important to protect our actual frame payloads, the metadata (channel number and timestamp) can also leave us vulnerable to attackers. Encrypting our metadata in this outer layer will help ensure that attackers cannot determine which channel a frame belongs to or infer transmission patterns for side door attacks. This metadata was encrypted using AES-CBC with a system-wide master key.
- **Using Randomness To Enhance Security:** Similar to how we utilized a random nonce in the inner layer to encrypt the inner layer, we used a random 16-byte IV to ensure that identical inputs still produced unique ciphertexts to prevent attackers from recognizing patterns in data.
- **Authentication and Verification:** AES-CBC only provides encryption for data secrecy, meaning an attacker can modify the ciphertext without being detected. To mitigate this, we used an Encrypt-Then-Mac (EtM) approach. In this case, HMAC-SHA256 is computed over the IV and ciphertext. Upon decryption, the HMAC is verified before the plaintext is decrypted. This ensures that the tampered messages are rejected. We can also prevent replay attacks (see next bullet).
- **Using HMAC and Timestamps To Prevent Replay Attacks:** A replay attack occurs when an attacker gets their hands on a previously sent encrypted frame and sends it to the receiver. Because we have encrypted our metadata (timestamps), they will not be able to modify our timestamps. However, because we utilize monotonically increasing timestamps, the decoder is able to securely store the timestamp of the last accepted frame and compare it against incoming frames to prevent timestamp rollback attacks. This ensures that frames

will only be decrypted for active subscriptions, with HMAC-SHA256 verifying integrity and monotonically increasing timestamps preventing replay attacks.

- **Padding Compliance:** Since AES-CBC requires data to be in 16-byte blocks, we ensured that the appropriate padding was applied to each plaintext before encryption.

Outer Layer Encryption Process: The encryption process continues in encryption.py where the final output from the inner layer encryption process is further encryption alongside the metadata (channel number and timestamp). This process secures the entire structure using a system-wide master key. Prior to encryption, a random 16-byte IV is generated using `os.random(16)`, ensuring that identical plaintexts produce different ciphertexts. The AES-CBC encryption process then encrypts previously encrypted inner layer along with the metadata. Once that is done, an HMAC-SHA256 authentication tag is computed over [AES-CBC IV] + [AES-CBC Ciphertext] in authentication.py to ensure that any modification to the encrypted frame is detected before the decryption process begins. The final structure is transmitted to the decoder for decryption.



Figure 2: Inner-Layer Encryption Strategy

5.2.2 The Decryption Process For Authorized Viewing

The decoder is responsible for securely receiving, verifying, and decrypting frames that were encrypted previously using our two-layer encryption strategy. It ensures:

- Only actively subscribed channels for the decoder can decrypt frames
- Replay attacks are prevented through timestamp verification
- Tampered frames are detected and rejected via HMAC authentication
- Both metadata and payloads remain confidential through AES decryption

Decoder Initialization

To initialize the decoder, there are a few steps we will need to follow to properly configure the bin file: To create the bin file needed to flash the board, we will run the following function in the root folder:

Linux and Powershell:

```
docker run --rm -v .\deadbeef_build:/out -v .\decoder:/decoder -v
.\global.secrets:/global.secrets:ro -e DECODER_ID=0xdeadbeef decoder
```

This will save the max78000.bin within the deadbeef_build folder in the root directory. To then flash that bin folder onto the board along with the global secrets, you will need to run the command:

Linux:

```
python -m ectf25.utils.flash ./deadbeef_build/max78000.bin
/dev/tty.usbmodem[port]
```

Powershell:

```
python -m ectf25.utils.flash .\deadbeef_build\max78000.bin COM[port]
```

For Windows:

For Window users, because our decoder build requires us to use Unix-style line endings instead of DOS-style line endings, we need to use the `dos2unix` command on the `build.sh` file within the decoder to change the DOS-style to a Unix-style. To do this, we are going to download the `dos2unix` command from the `choco` library by running: `choco install dos2unix -y`. Then, once downloaded, enter the decoder and run the command: `dos2unix build.sh`. This should change the line-ending styling of the build file, allowing windows to build the decoder as well.

Decoder Functionalities

After initialization, the decoder functions with the following keys, cryptography library, and subscription and decryption process.

- **Decoder Key Management:** To begin the process of securely decrypting the frames, the decoder stores and retrieves cryptographic keys required for decryption. These keys include the AES-256 master key (for outer decryption), per channel AES-128 keys (for inner decryption), and the HMAC signing key for authentication. The decoder stores its cryptographic keys inside `decoder_secrets.c` and accesses them via `decoder_secrets.h`. Relevant keys include:
 - **Master Key: AES-256:** Used for decrypting the outer layer (AES-CBC)
 - **Per Channel AES-128 Keys:** Used for decrypting the inner layer (AES-GCM)
 - **HMAC-SHA256 Authentication Key:** Used to verify the integrity of the received frame
- **WolfSSL:** Unlike our encryption process, our decryption process primarily happens in an embedded environment. `decoder.c` is flashed onto an embedded device meaning all cryptographic operations must be highly optimized for real-time processing. WolfSSL allows the decoder to verify, decrypt, and extract data from an incoming encrypted frame in a more efficient manner than other cryptographic packages and libraries.
- **Channel Subscription Process** The Decoder maintains a list of unauthorized channels based on the user's subscriptions. It provides an interface for the Host Computer to retrieve the list of available channels.
 - **Retrieving keys:** Subscription tokens are generated on a secure system before being sent to the decoder where it is stored securely for persistent access. In our system, tokens are created in `gen_subscription.py`. During this process, a secure signing key is loaded from `gen_secrets.py`, which signs the token to ensure only authorized decoders can validate it. It's then verified in `inspect_subscriptions.py` before being stored and used in `decoder.c` for access control before decryption.
 - **Subscription Update:** After decrypting the metadata in the outer layer, the decoder verifies the subscription request by checking the signature, decoder ID and the channel that the update is requesting. If both signature and decoder ID match and the requested channel is not the emergency channel, the subscription is updated in the subscription list.
- **The Decryption Process:**
 - **Receipt and Extraction:** The decoder receives an encrypted frame which consists of the final payload described in the outer encryption layer section. From this payload, the decoder extracts
 - * The IV (16 bytes) for AES-CBC decryption
 - * The cipher text (which contains both the metadata and the encrypted payload)
 - * The HMAC SHA-256 authentication tag
 - **Authentication:** Before decrypting, the decoder recomputes HMAC-SHA256 using the stored signing key and verifies it matches the received HMAC signature. This ensures that the frame has not been tampered with.
 - **Outer Layer Decryption:** If the signature is valid, the decoder retrieves the master key to decrypt the outer layer. This extracts the meta data and the inner encryption output.
 - **Decoder ID Verification:** Once metadata is extracted from the outer layer, the decoder checks if decoder ID matches.
 - **Subscription Verification:** If decoder ID matches, the decoder checks if the requested channel has a valid subscription.
 - **Timestamp Verification:** The decoder checks if the current timestamp is in the range of the start and end time of the subscription to verify that the subscription is active. Additionally, it also verifies that the timestamp is strictly increasing by comparing the current timestamp to the last received timestamp to prevent replay attacks.
 - **Inner Layer Decryption and Reply:** The decoder then extracts the 12-byte nonce from the encrypted payload and decrypts the inner layer of the active channel. It then returns the decrypted frame.

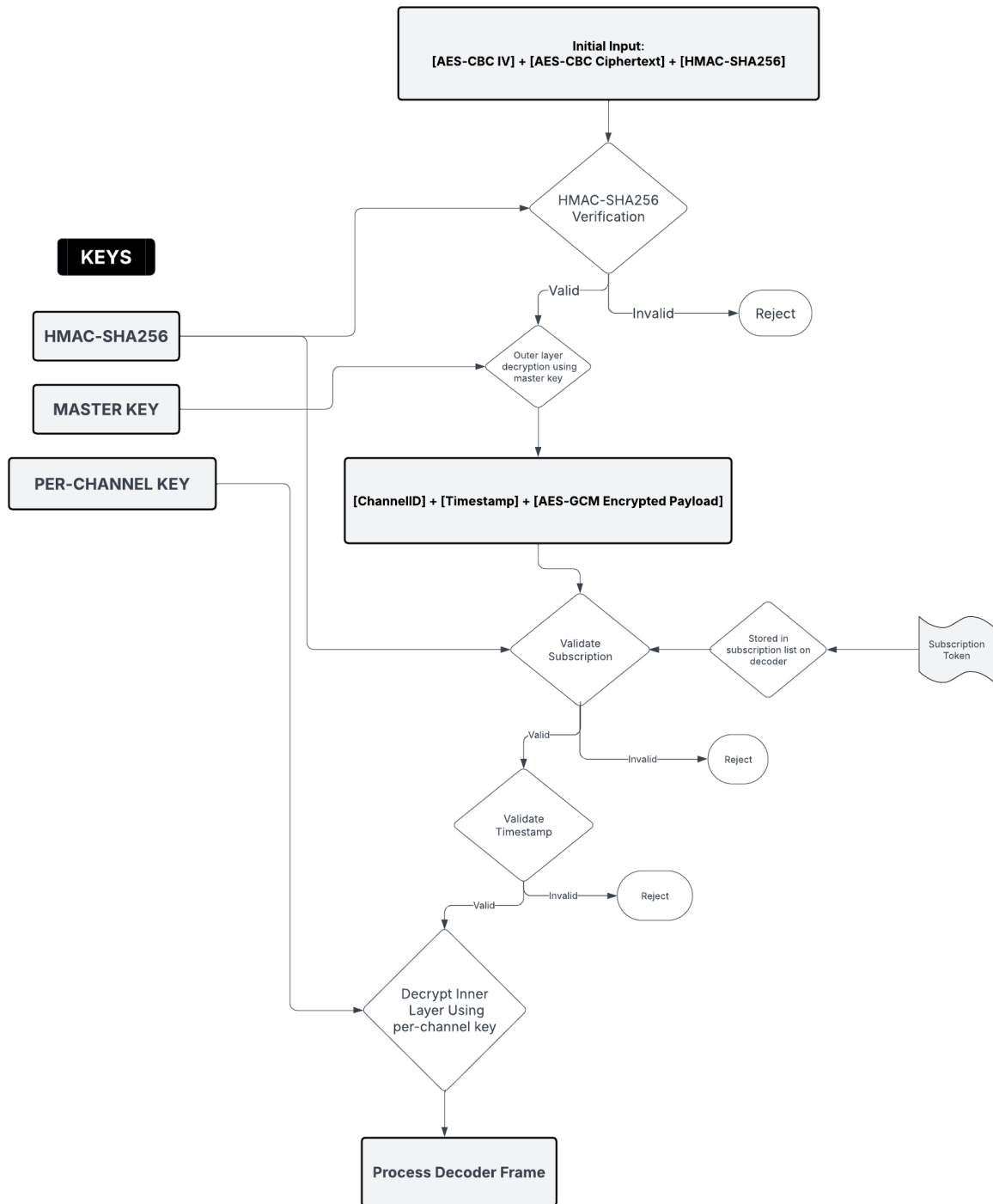


Figure 3: Inner-Layer Encryption Strategy

6 Hardware Security Protocols

6.1 Memory Storage and Protection

Securing encryption keys is critical to preventing unauthorized access and exploitation of sensitive data. The Memory Protection Unit (MPU) allows us to enforce access control policies on memory regions, reducing the risk of key leakage, buffer overflows, and other attacks.

In this system, we have 9 encryption keys that must remain secure when not in use. However, since this region of memory is not dedicated, we need a robust memory protection strategy to prevent unauthorized access while ensuring that the keys can still be utilized in our encryption protocols.

Key Security Considerations For Memory Protection

Security Threat	Mitigation	Implemented Protocol
Buffer Overflow	An attacker overwrites memory beyond allocated buffers, potentially modifying or gaining access to encryption keys	Establish MPU guard regions that limit access to key storage and detect unauthorized overflows reducing the risk of memory corruption and key leakage.
Stack and Heap Overflow	An attacker overflows heap or stack memory to corrupt critical data, including encryption keys.	Restrict access between the heap and the stack and utilize memory allocators to prevent large, unchecked memory allocations.
Code Injection Attacks	Injection of malicious code into executable memory regions in an attempt to extract keys.	Prevent execution in key and data storage areas.

Table 2: Security Threats and Mitigations

Memory Protection Unit Our primary strategy involved the use of guard regions (privileged memory areas) to protect buffer overflow from the stack or the heap into the region where the encryption keys are stored. No supervisor call is needed to retrieve the keys because the keys themselves are not stored in a protected region.

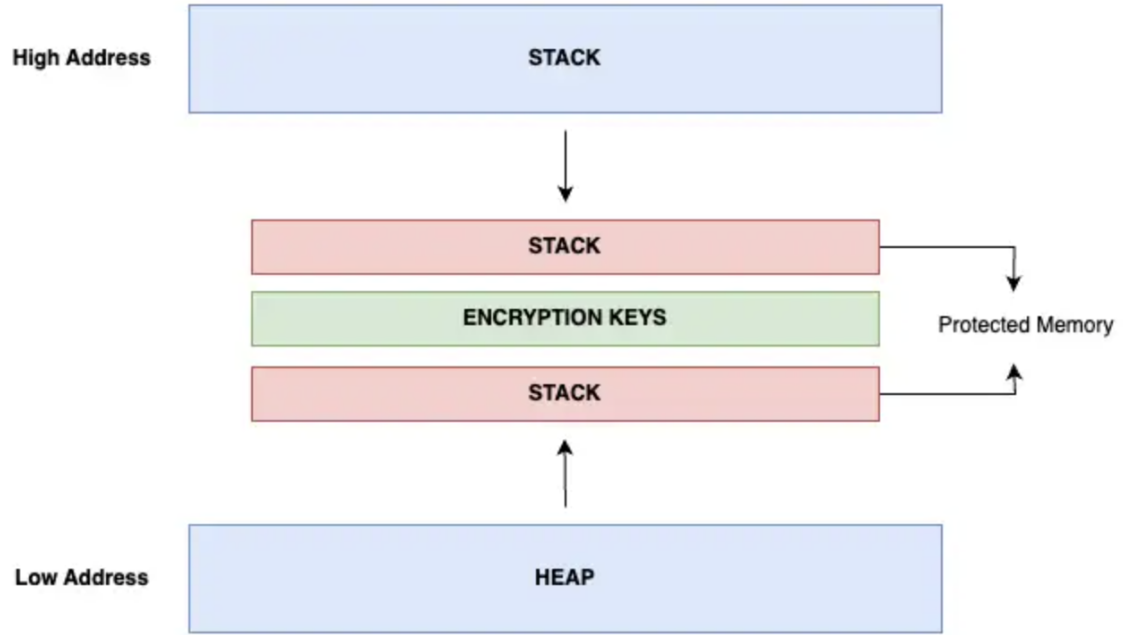


Figure 4: Inner-Layer Encryption Strategy

Memory	Access Type	Purpose
Stack	Read/Write	Standard execution stack
Guard Region I	No access (MPU protected)	Prevents stack overflow from corrupting encryption keys.
Encryption Key Storage	Read/Write	Stores keys in a dedicated un-privileged region.
Guard Region II	No access (MPU protected)	Prevents heap overflow from corrupting encryption keys.
Heap	Read/Write	Dynamic memory allocations.

Table 3: MPU Permissions and Purpose

Table B3-15 Access permissions field encoding

AP[2:0]	Privileged access	Unprivileged access	Notes
000	No access	No access	Any access generates a permission fault
001	Read/write	No access	Privileged access only
010	Read/write	Read-only	Any unprivileged write generates a permission fault
011	Read/write	Read/write	Full access
100	UNPREDICTABLE	UNPREDICTABLE	Reserved
101	Read-only	No access	Privileged read-only
110	Read-only	Read-only	Privileged and unprivileged read-only
111	Read-only	Read-only	Privileged and unprivileged read-only

Figure 5: configurations for the MPU and what level of privilege each mode allows

6.1.1 UART Bounds Checking

UART communication involves receiving arbitrarily sized data from external sources, which if unchecked could be malicious or malformed. If the header size is too large, it could overflow the memory causing memory corruption or code execution vulnerabilities. To solve this problem, we perform a bounds check on read_HEADER. This prevents buffer overflow by enforcing a size limit and rejects oversized and potentially malicious headers.

7 Conclusion

Our encryption and authentication system has been systematically designed to provide a secure, scalable, and efficient solution for protecting sensitive communications while implementing all of the functional and security requirements for the competition.

By implementing AES-GCM for frame payload encryption, AES-CBC for metadata protection, and HMAC-SHA256 for message integrity verification and executing sanity checks, our design safeguards against critical security threats that we took into consideration throughout this process, including replay attacks, data tampering, and brute force decryption attempts.

To ensure long-term security and prevent cryptographic vulnerabilities, we have employed HKDF-derived per channel keys, which help to enhance forward secrecy and mitigate risks associated with key reuse. Additionally, our memory protection unit (MPU) integration enforces stringent access protocols to prevent unauthorized access to encryption keys and protect against memory-based attacks such as buffer overflows.

Our subscription management system leverages HMAC-verified authentication tokens to ensure that only authorized devices can access and decrypt messages. Furthermore, our monotonic timestamp validation works to mitigate the risk of replay attacks by guaranteeing that only valid and recent messages are processed.

Finally, in order to optimize for high-performance in an embedded environment, we utilized WolfSSL, an industry-standard cryptographic library designed for performance in embedded environments where resources are constrained. This allowed our team to deliver a solution that meets the rigorous security protocols asked of our system without compromising efficiency.

Our design adheres to the functional and security requirements outlined in the MITRE eCTF competition, demonstrating a proactive approach to security in embedded systems. We believe that our system

is well positioned to meet the needs of organizations seeking secure, resilient, and efficient communication infrastructures.

A Terminology and Notation

Abbreviations

- **AES** - Advanced Encryption Standard.
- **AES-CBC** - Advanced Encryption Standard - Cipher Block Chaining.
- **AES-GCM** - Advanced Encryption Standard - Galois/Counter Mode.
- **API** - Application Programming Interface.
- **CTR** - Counter Mode.
- **eCTF** - Embedded Capture the Flag.
- **HKDF** - HMAC-based Key Derivation Function.
- **HMAC** - Hash-based Message Authentication Code.
- **IKM** - Input Keying Material.
- **IV** - Initialization Vector.
- **MPU** - Memory Protection Unit.
- **OKM** - Output Keying Material.
- **PRK** - Pseudorandom Key.
- **SHA** - Secure Hash Algorithms.
- **SSL** - Secure Socket Layer.
- **UART** - Universal Asynchronous Receiver-Transmitter.